OneTapp Consulting Hackathon

AI Engineer Assessment

OneTapp AI Retail Analytics Assistant
AI-Powered Conversational Business Intelligence System for Retail Analytics

Submitted By
Name: A Mohammed Mudassir
Roll Number: 1CR22EC001
College: CMR Institute of Technology

Domain
AI Engineer

Technology Stack

- Python
- Groq LLM
- SQLite
- Streamlit
- Pandas

Project Summary

This project enables business users to ask retail analytics questions in natural language, automatically converts them into SQL queries using an LLM, retrieves relevant data from a SQLite database, and returns concise business insights through a conversational interface.

Project Links

Live Demo : https://onetapp-ai-analytics-assistant-qhgbdb5mhjrpjebnhansk3.streamlit.app/

GitHub Repository : https://github.com/amohammedmudassir/OneTapp-AI-Analytics-Assistant

Submitted for: *OneTapp Consulting Hackathon - AI Engineer Assessment*

18 July 2026

Section A : Problem Decomposition

Understanding the Business Problem

Business teams often rely on data analysts to answer questions related to sales performance, inventory movement, pricing, promotions, and regional trends. Although dashboards provide useful visualizations, users still depend on technical teams whenever they need customized analysis or comparisons. This slows down decision-making and increases dependency on data analysts.

To solve this problem, I built an **AI-powered Retail Analytics Assistant** that allows business users to interact with retail data using natural language. Instead of manually exploring dashboards or writing SQL queries, users can simply type their questions through the Streamlit interface. The assistant automatically converts the request into SQL, retrieves the required information from the database, and presents the results in an easy-to-understand business format.

The assistant is designed to answer common business questions such as:

- Sales performance across different regions
- Inventory availability and stock movement
- Promotion effectiveness
- Product and category comparisons
- Pricing and discount analysis

Some example questions include:

- Which region sold the highest number of units?
- Did promotions improve sales performance?
- What is the average product price by category?

Instead of displaying only raw SQL query results, I wanted the assistant to generate meaningful business insights. After retrieving the required data from the SQLite database, the results are passed to the LLM, which produces a concise business summary and recommendation based only on the retrieved records. This enables business users to understand analytical results quickly without requiring any knowledge of SQL or database systems.

Section B : AI Solution Approach

To implement this solution, I used **Python** as the primary programming language along with **Streamlit** for the user interface, **SQLite** as the analytical database, **Pandas** for data pre-processing, and the **Groq LLM API** for natural language understanding and reasoning.

Before building the assistant, I converted the **Retail Store Inventory Forecasting** dataset from Kaggle into a SQLite database using Pandas (load_data.py). I selected this dataset because it contains realistic retail business information such as product categories, regional sales, inventory levels, pricing, promotions, seasonal data, and demand forecasts. Since all the required analytical attributes were available in a single dataset, it allowed the assistant to answer a wide variety of business questions without combining multiple data sources.

When a user submits a question through the Streamlit application, it is first sent to **sql_agent.py**, where the Groq LLM generates an SQLite query using the database schema provided in the system prompt. The generated SQL query is then executed through **database_query.py**, and the returned results are converted into a Pandas DataFrame. Finally, the DataFrame is passed to **insight_generator.py**, where

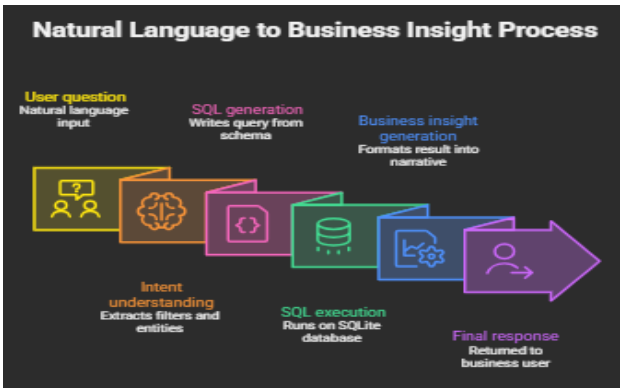
the LLM generates a concise business explanation and recommendation based only on the retrieved data.

While designing the solution, I considered different AI architectures. Since this project works entirely with structured retail data stored in relational tables, I intentionally did not use Retrieval-Augmented Generation (RAG). RAG is more suitable for retrieving information from unstructured sources such as PDFs, manuals, or documents. For this use case, converting natural language into SQL queries provided a simpler, faster, and more reliable solution while keeping the overall architecture lightweight and easy to maintain.

Agentic Construct

Although the project uses a lightweight architecture, it follows an agentic workflow where each component performs a dedicated responsibility before passing its output to the next stage. Instead of relying on a single AI response, the system separates natural language understanding, SQL generation, database interaction, validation, and business insight generation into independent modules.

Workflow:



Each component has a specific responsibility:

- **Streamlit (app.py)** – Collects the user's business question and displays the generated results.
- **SQL Agent (sql_agent.py)** – Uses the Groq LLM to convert natural language into SQLite queries.
- **SQLite (retail.db)** – Stores the structured retail analytics dataset.
- **Database Executor (database_query.py)** – Executes SQL queries and retrieves analytical results safely.
- **Business Insight Generator (insight_generator.py)** – Converts SQL query results into concise business summaries and recommendations.
- **Validation Layer (validator.py)** – Ensures that only relevant retail analytics questions are processed, reducing invalid queries and improving reliability.

This modular architecture keeps the implementation simple, maintainable, and easy to extend while ensuring that every business insight is generated from actual database results rather than assumptions.

Section C : Data Interaction Strategy

Data Interaction Strategy

To make the assistant work with structured retail data, I first converted the Kaggle CSV dataset into a SQLite database using **Pandas** in load_data.py. This created a local database (retail.db) containing all the sales, inventory, pricing, promotion, and regional information required for analysis.

When a user asks a business question through the Streamlit interface, the question is first sent to the Groq LLM. In my implementation, the LLM does not answer the question directly. Instead, it generates an SQLite query based on the user's request by referring to the database schema provided in the prompt (sql_agent.py).

Once the SQL query is generated, it is executed using Python's **sqlite3** library inside database_query.py. The query result is returned as a Pandas DataFrame. Instead of displaying only raw database values, I pass this DataFrame back to the LLM through insight_generator.py, where it generates a short business explanation and recommendation based only on the retrieved data.

For analytical operations, I relied on SQL itself instead of performing calculations in Python. Queries generated by the LLM use SQL operations such as:

- SUM() for total sales or inventory
- AVG() for average pricing
- GROUP BY for regional and category analysis
- ORDER BY for ranking
- LIMIT for top-performing results
- WHERE for filtering specific records

Using SQL for aggregation keeps the implementation simple, efficient, and scalable while allowing the LLM to focus only on natural language understanding and business explanation.

Section D : Limitations & Risks

Limitations & Risks

While implementing the assistant, I identified a few practical challenges and handled them directly in the code.

1. Incorrect SQL Query Generation

Since SQL queries are generated by the LLM, there is a possibility that the model may produce an invalid or incorrect SQL statement.

How I handled it:

In database_query.py, I wrapped SQL execution inside a **try-except** block. If the generated SQL fails to execute, the application catches the exception and displays a user-friendly message instead of crashing the application.

2. Hallucinated Responses

Initially, I observed that the LLM sometimes generated additional information that was not present in the database (for example, mentioning percentages that were never returned by the SQL query).

How I handled it:

I modified the prompt inside insight_generator.py to explicitly instruct the model to generate business insights **only from the SQL query results** and never invent numerical values or assumptions beyond the retrieved data.

3. Data Misinterpretation

Another challenge occurred when users asked questions outside the supported business domain or asked incomplete questions.

For example:

Who sold the highest units?

The question does not specify whether the user is referring to a **region**, **product**, **store**, or **category**.

How I handled it:

I implemented a simple validation layer (validator.py) that checks whether the question belongs to the retail analytics domain before sending it to the LLM. If an unsupported question is detected, the assistant politely informs the user that it can only answer retail analytics questions. This prevents unnecessary SQL generation and improves the overall user experience.

Section E : Design Trade-Off

While designing this project, I intentionally chose **simplicity over flexibility**.

Since the assistant works entirely with structured retail data, I avoided introducing additional components such as Retrieval-Augmented Generation (RAG), vector databases, or multi-agent frameworks. Although these approaches provide greater flexibility for handling unstructured information, they also increase implementation complexity, resource usage, and maintenance effort.

Instead, I designed a lightweight architecture where the LLM is responsible only for understanding the user's question, generating SQL queries, and producing business-friendly insights. All analytical computations are performed directly by the SQLite database, making the overall workflow simpler, faster, and easier to debug.

This design decision allowed me to build a solution that is reliable for structured retail analytics while keeping the implementation clean, maintainable, and aligned with the problem requirements. If the project were extended in the future to support unstructured documents or multiple business domains, additional components such as RAG or vector databases could be incorporated without changing the core architecture.

Project Results (Screenshots):